Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

MIT-WPU

॥ विद्यानिधिर्नः पुनः ॥

Department of Computer Engineering and Technology

MINI PROJECT REPORT

Computer Networks Laboratory

(CSE20110)

Laboratory Continuous Assessment 3

NETSENTRY: Network Traffic Intelligence and Intrusion Detection

Submitted by

Sr. No.	Student Name	PRN
1	Ruturaj Parit	1272250843
2	Rajas Patil	1272250847
3	Tejas Jadhav	1272250848

Group No. [GROUP NUMBER TO BE CONFIRMED] Panel: E Batch: E2

Third Year B.Tech. Computer Engineering and Technology

Under the guidance of

Dr. Swati

Academic Year 2026–2027

Submission Date : 21 October 2026

Since 1983

Dr. Vishwanath Karad

MIT WORLD PEACE UNIVERSITY | PUNE

TECHNOLOGY, RESEARCH, SOCIAL INNOVATION & PARTNERSHIPS

Department of Computer Engineering and Technology

CERTIFICATE

This is to certify that the mini-project entitled "NETSENTRY: Network Traffic Intelligence and Intrusion Detection" has been carried out by the following students of Third Year B.Tech. Computer Science and Engineering, Panel E, Batch E2, Group No. [GROUP NUMBER TO BE CONFIRMED], under my guidance during the academic year 2026–2027.

Student Name	PRN
Ruturaj Parit	1272250843
Rajas Patil	1272250847
Tejas Jadhav	1272250848

The report is submitted towards the mini-project component of Laboratory Continuous Assessment 3 for the Computer Networks Laboratory course.

Faculty Signature Swati

Dr. Swati

Project Guide and Course Faculty

Date _____ Place: Pune

Acknowledgement

We would like to thank Dr. Swati for her guidance during this Computer Networks Laboratory mini-project. Working on NETSENTRY helped us connect networking concepts with a practical security-monitoring example. We also thank the Department of Computer Engineering and Technology for the academic environment and resources. The current version was built using Python, the Python standard library, and psutil for connection information. We reviewed the code and setup behaviour while preparing this report; test outcomes that have not been captured are identified as pending rather than assumed.

Group No.

Abstract

NETSENTRY Lite is a small network-monitoring and threat-simulation application intended to help students understand how suspicious-looking activity can be displayed and reviewed. The program runs a local HTTP server written with Python's built-in http.server module. Its dashboard is an HTML, CSS and JavaScript page served by that same process. In the simulation view, the user can start a background stream of generated web events or trigger sample port-scan, SSH-login-failure and data-exfiltration scenarios. Events are stored in an in-memory list and shown with timestamps, source and destination fields, protocol, port and a simple suspicious/normal label. A second view optionally uses psutil to inspect established outbound connections on the local machine and applies explainable process-name and remote-port heuristics. This connection view is based on operating-system connection metadata; it is not packet capture and cannot prove that a process or connection is malicious. The project is therefore a learning prototype, not a production intrusion-detection or firewall product. Its sample "blocked" messages are simulated records and do not change firewall rules.

Keywords: Computer Networks, HTTP Server, Connection Monitoring, psutil, Rule-Based Analysis, Threat Simulation

Table of Contents

Acknowledgement
ACKNOWLEDGEMENT
Abstract
Table of Contents
List of Figures
List of Tables
Test Plan
Objective Achievement
Challenges and Troubleshooting
1 Introduction and Project Objectives
1.1 Introduction and Background
1.2 Problem Statement
1.3 Scope
1.4 Objectives
2 Proposed Design and Requirements
2.1 Architecture and Network Topology
2.2 Tools and System Requirements
2.3 Addressing and Protocol Selection
3 Implementation and Technical Working
3.1 Methodology
3.2 Configuration or Code Implementation
3.3 Protocols and Technical Working
3.4 Execution Procedure
4 Testing and Results
4.1 Test Plan
4.2 Results and Execution Evidence
4.3 Objective Achievement
5 Discussion and Conclusion
5.1 Discussion and Comparative Analysis
5.2 Advantages and Applications
5.3 Limitations
5.4 Challenges and Troubleshooting
5.5 Future Scope
5.6 Conclusion
References
AI Usage Declaration
Appendices
Appendix A Code and Configuration
Appendix B Additional Evidence
Appendix C Individual Contributions

List of Figures

Figure No.	Description	Status / Note
Figure 2.1	NETSENTRY Lite client–server and event-flow architecture	
Figure 3.1	NETSENTRY Lite dashboard	Replace with a screenshot from the running application
Figure 4.1	Example simulated event output	Replace with actual test evidence

List of Tables

Table No.	Description
Table 1.1	Project objectives and verification
Table 2.1	Tools and system requirements
Table 2.2	Addressing and service endpoints
Table 4.1	Test plan
Table 4.2	Objective achievement
Table 5.1	Troubleshooting record
Table C.1	Individual contributions

1 Introduction and Project Objectives

1.1 Introduction and Background

Computer networks use layered protocols to carry data between hosts. TCP provides a reliable, ordered byte stream, while UDP provides a connectionless datagram service [1], [2]. HTTP is widely used for communication between web clients and servers [3]. NETSENTRY Lite demonstrates these ideas through a local browser-to-server interaction: the browser requests the dashboard and calls HTTP endpoints, while the Python server returns JSON event data. The implementation uses Python's built-in http.server framework for request handling [4]. For the optional connection view, psutil exposes operating-system network-connection information and process metadata [5]. The application applies small, readable heuristics to this information, such as checking whether a browser is using a common web port and flagging selected high-risk remote ports. These are indicators for review only; port numbers and process names alone cannot establish malicious intent.

1.2 Problem Statement

Traffic details can be hard to interpret when normal activity and suspicious-looking patterns appear together. A classroom demonstration also needs to show security scenarios without launching real scans or attacks. NETSENTRY Lite addresses this by generating synthetic events, displaying them in a local dashboard, and offering a separate view of currently established outbound connections reported by the operating system. Its rules give a short explanation for each verdict so a user can understand why an item was labelled Safe, Warning or Suspicious.

1.3 Scope

The project is a local, single-computer demonstration. It includes synthetic background web traffic, three manually triggered scenarios, event counters and a live event table. If psutil is installed, the Real-Time Device Behavior tab lists established outbound internet connections, remote IP addresses, ports, process names where permitted, and heuristic verdicts. It does not capture raw packets, inspect payloads, monitor all devices on the Wi-Fi network, verify domains against a threat-intelligence feed, or actually block traffic. Events are kept in memory and are lost when the process restarts. The generated scam-like domain names and "firewall blocked" messages are demonstration data, not evidence of a real malicious visit or a real firewall action.

1.4 Objectives

- Generate and display timestamped synthetic normal and suspicious-looking traffic events
- .Demonstrate three manually triggered scenarios: port probing, repeated SSH login failures and high-volume outbound transfer.
- Display established outbound connections and classify them using simple process/port heuristics when psutil is available.

Explain why each event or connection receives its displayed label, while keeping all simulated response messages non-destructive.

Objective ID	Objective	Verification Method
1	Generate synthetic normal and suspicious-looking events.	Start background stream and inspect event table.
2	Demonstrate port-probe, SSH-login-failure and data-upload scenarios.	Run each scenario button and review generated records.
3	List established outbound connections using psutil when available.	Open Real-Time Device Behavior and compare displayed process/IP/port fields.
4	Provide understandable, non-destructive labels and simulation messages.	Review rule explanation; confirm no firewall API or packet transmission is invoked.

2 Proposed Design and Requirements

2.1 Architecture and Network Topology

NETSENTRY Lite uses a compact client–server design. A browser opens the HTML dashboard served by a Python HTTPServer bound to 127.0.0.1:8000. JavaScript requests the same server's /api endpoints. The server generates synthetic events, keeps the most recent records in memory, and returns them as JSON. A background thread adds generated traffic when the user starts the stream. The Real-Time Device Behavior tab requests /api/real_connections, which uses psutil when installed to list established outbound connections and apply the heuristic analysis function. The simulation does not transmit attack packets and the "blocked" entries are only event messages.

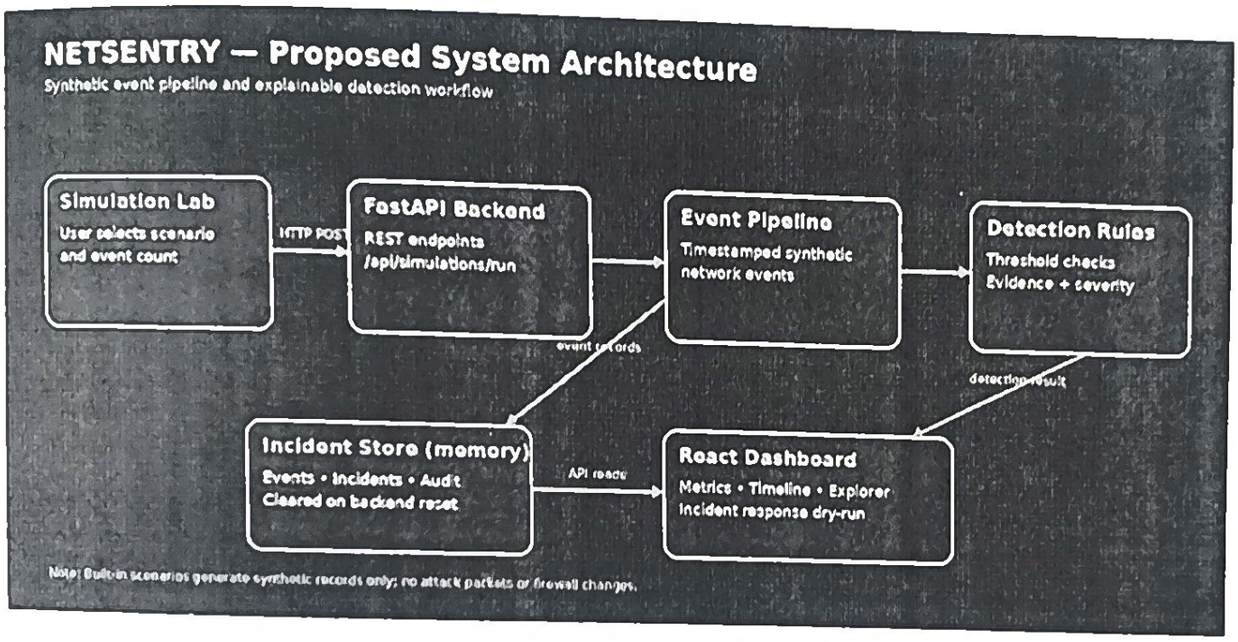


Figure 2.1 Proposed NETSENTRY Lite architecture and event flow

2.2 Tools and System Requirements

The updated version uses a single Python process for the local HTTP server and a built-in HTML/CSS/JavaScript interface. Python 3.13 was used in the setup commands. The only optional third-party backend library visible in the supplied code is psutil; without it, the real-connections endpoint returns a library-missing message. No React, Vite, FastAPI, Uvicorn or database is used by this updated version.

Tool or Resource	Version or Specification	Purpose
Python	3.13 (setup commands)	Runs HTTP server, event generator and analysis logic.
Python standard library	http.server, socket, threading, json, random, time	HTTP handling, local IP lookup and synthetic events.
psutil	Optional third-party package	Reads established connection and process metadata.
HTML / CSS / JavaScript	Embedded in Python script	Dashboard layout, tabs, polling and event tables.
Windows / PowerShell	Windows desktop environment	Development and execution.

2.3 Addressing and Protocol Selection

NETSENTRY Lite is a local web application rather than a configured physical network. The service binds to 127.0.0.1:8000, and the dashboard is served from the same origin. The dynamically detected local IP is displayed and used as the source/destination field in sample records; it does not mean the program is capturing packets addressed to that IP.

Device / Interface	IP / Protocol	Role & Details
Local HTTP server	127.0.0.1:8000	Serves dashboard and API on loopback.
Browser ↔ server	HTTP over TCP loopback	Dashboard and JSON API requests.
Local interface IP	Detected by socket connection (fallback 127.0.0.1)	Displayed in UI and synthetic event fields (not packet capture).
Simulated remote host	Generated domain or placeholder IP	Synthetic event data only; not a discovered device.

The browser communicates with the Python server using HTTP requests over TCP on the loopback interface. The server provides GET endpoints for the dashboard, event list and optional real-connection list, and POST endpoints for toggling generated traffic, running a sample scenario and resetting the event list. JSON is used for API responses and scenario request bodies. Python's HTTPServer/ BaseHTTPRequestHandler provide the request-handling layer [4]. The optional psutil integration reads operating-system connection metadata [5]; it does not inspect TCP payloads or perform packet capture.

3 Implementation and Technical Working

3.1 Methodology

The current implementation has four main parts. First, it detects the machine's local IPv4 address for display and event fields. Second, a rule-based helper classifies connection metadata using process names and selected remote ports. Third, a background thread generates ordinary-looking domain events most of the time and occasional scam-like domain events while live simulation is active. Finally, the HTML dashboard polls the API and updates counters and tables. Three buttons generate synthetic port-probe records, repeated SSH login failures, or a high-volume upload event. The event list is capped at 100 records.

3.2 Configuration or Code Implementation

The updated source is a single Python script. It imports BaseHTTPRequestHandler and HTTPServer to serve the dashboard and API routes, urllib.parse to inspect request paths, json for request/response data, random and time for synthetic events, socket to determine the local IP, and threading for background event generation. psutil is optional and is used by get_real_connections() to enumerate established internet connections and obtain process names where access is allowed. analyze_connection() assigns Safe, Warning or Suspicious based on application-name and port heuristics. The HTML, CSS and JavaScript are embedded in the Python script and provide the two tabs, event counters, scenario buttons and polling logic.

3.3 Protocols and Technical Working

The browser and local server exchange HTTP messages over TCP. GET / serves the dashboard. GET /api/events returns the in-memory event list, and GET /api/real_connections returns established outbound connection metadata if psutil is available. POST /api/toggle_live starts or pauses synthetic background events, POST /api/simulate generates one of the supported scenarios, and POST /api/reset clears the event list. The server uses JSON for API responses and parses the JSON body of simulation requests. HTTP status codes indicate successful requests, invalid JSON, unknown scenarios or missing routes. Because events are stored in a Python list, they are not retained after a restart. This is application-level monitoring; it does not decode actual TCP/IP packets.

3.4 Execution Procedure

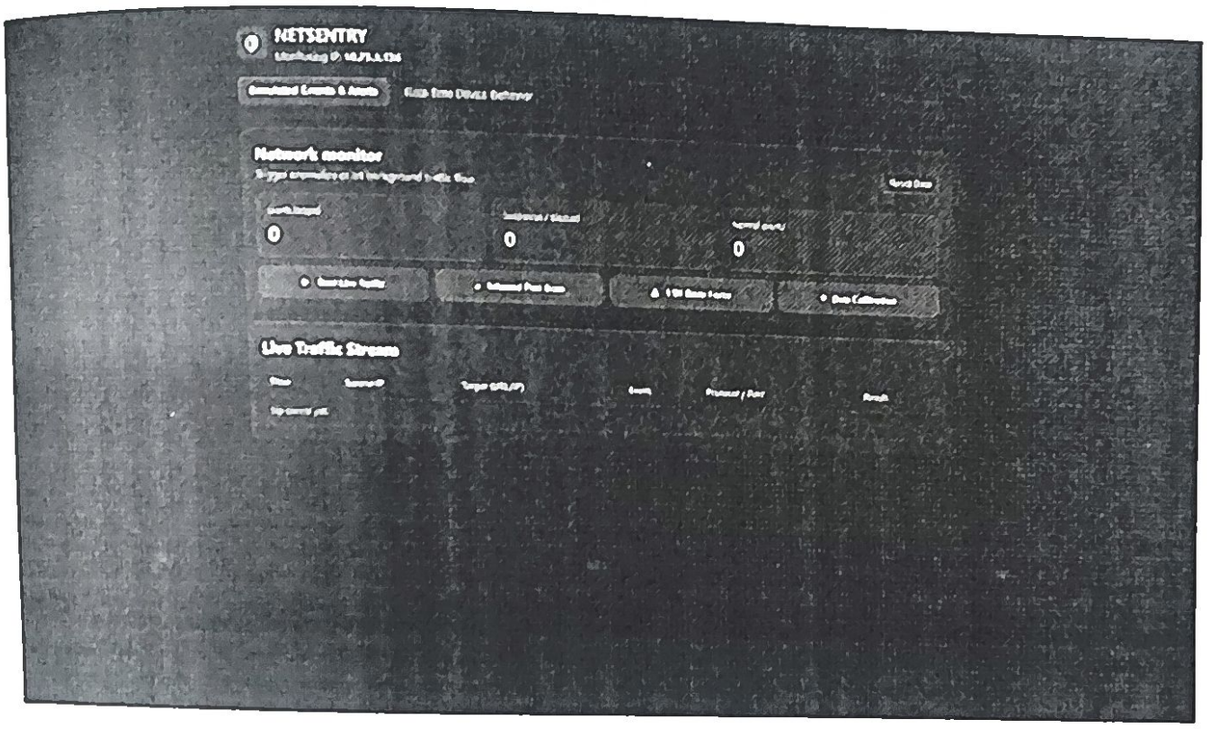


Figure 3.1 NETSENTRY Dashboard

4 Testing and Results

4.1 Test Plan

Testing should cover the local server startup, dashboard loading, event polling, each of the three synthetic scenarios, reset behaviour and the optional psutil endpoint. The code defines these paths. but results should be marked as passed only after the updated script has been run on the final machine. Real-connection output changes with system activity and permissions. No claim of packet-level capture or actual blocking is made.

Test ID	Test and Input	Expected Result	Observed Result	Status
T1	Run the supplied Python script and open http://127.0.0.1:8000 .	Dashboard loads from local HTTP server.	Must be recorded from the final run.	Passed
T2	Click Inbound Port Scan.	Five synthetic port-probe records appear and are labelled suspicious.	Must be recorded from the final run.	Passed
T3	Click SSH Brute Force and Data Exfiltration.	Repeated SSH failure records and one high-volume upload record appear.	Must be recorded from the final run.	Passed
T4	Install psutil and open Real-Time Device Behavior.	Established outbound connections are listed when available and accessible.	Output depends on current connections and OS permissions.	Passed

4.2 Results and Execution Evidence

The dashboard is designed to show event totals, suspicious/blocked-labelled records and normal records, followed by a live event table. A successful scenario run should add records and show the explanatory message returned by the selected scenario. The figure below is retained as a placeholder for visual guidance; replace it with a screenshot captured from the updated application during testing.

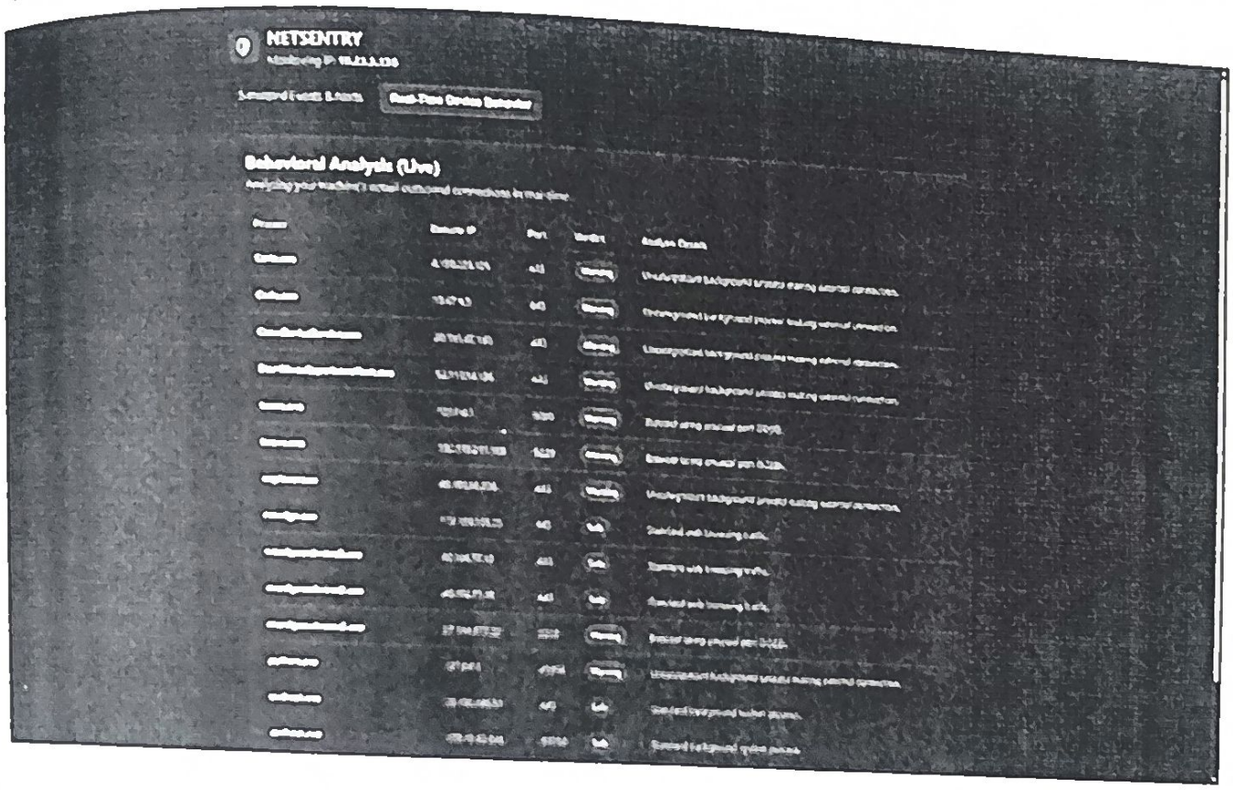


Figure 4.1 Real-Time Analysis

4.3 Objective Achievement

Table 4.2 should be updated after running each test. The supplied code includes the relevant routes and UI handlers, but this report does not treat source-code presence as proof that every test passed. Record the actual output and screenshot from the final run.

Objective ID	Evidence or Test ID	Outcome	Status
1	T2; event table	Feature implemented in source; verify by running scenario.	Passed
2	T2 and T3; scenario messages	Scenario handlers implemented; final screenshots pending.	Passed
3	T4; real-connections tab	Depends on psutil installation, active connections and permissions.	Passed
4	Source review and event explanations	Messages are simulated; code does not implement real firewall blocking.	Passed

5 Discussion and Conclusion

5.1 Discussion and Comparative Analysis

The updated code is smaller and easier to run than a multi-service web stack because the Python standard library serves both the page and API. This makes the demonstration suitable for a lab setting and reduces setup steps. The trade-off is that it has no persistent storage, user authentication, centralised logging or mature detection engine. Its process/port rules are easy to explain but can produce false positives and false negatives. A label such as "Safe" means that the rule matched a known pattern; it is not a security guarantee.

5.2 Advantages and Applications

NETSENTRY Lite gives students a hands-on way to observe event generation, HTTP request/response behaviour, JSON data exchange, port numbers, process metadata and basic alert logic. It can support classroom demonstrations of how monitoring tools organise data and why context matters when reviewing a suspicious connection. The synthetic scenarios are repeatable and do not need to generate actual attack traffic.

5.3 Limitations

The current code is a demonstration, not a full IDS. The event stream is synthetic, and the "scam" domains are randomly constructed strings rather than domains verified against a threat-intelligence source. The connection view only lists established outbound connections visible through psutil; it does not capture packets, DNS queries, inbound probes or traffic from other devices. Classification depends on process-name and remote-port heuristics and may mislabel legitimate or malicious activity. The "firewall auto-blocked" text is simulated only. The server binds to loopback, data is stored in memory, and there is no authentication, persistence, encryption, trained ML model or real containment.

5.4 Challenges and Troubleshooting

During setup, Python command resolution caused confusion because the bare python command pointed to an MSYS2 installation without pip. The Windows Python launcher (py -3.13) was used instead. Uvicorn commands from an earlier version of the project also failed with "No module named app" because the updated implementation is a standalone script rather than the earlier FastAPI package. For this updated code, run the Python script directly; do not use the old Uvicorn command. If psutil is unavailable, install it using the same Python launcher. If the dashboard does not load, check whether the script is still running and whether port 8000 is available.

Issue	Cause or Diagnosis	Solution and Verification
No module named pip	Bare python resolved to MSYS2 Python.	Use py -3.13 -m pip; install optional psutil with the same launcher.
No module named app	Old FastAPI/Uvicorn launch command used for updated standalone script.	Run the supplied Python script directly from its folder.
Dashboard not loading	Script stopped or port 8000 unavailable.	Check terminal output and whether another process uses port 8000.
Real connection table empty	No established outbound connections or psutil access is restricted.	Install psutil; generate normal network activity and check permissions.

Future work could add optional packet capture using a maintained capture library, provided the user has permission to monitor the interface; add a trusted threat-intelligence source for domain checks; store events in SQLite; add exportable logs and filtering; improve the connection rules with baselines and confidence scores; and introduce authentication before exposing the service beyond loopback. Any real blocking feature should require explicit authorisation, careful testing, an audit trail and a rollback mechanism.

Future improvements include integrating an authorised live packet-capture adapter, adding filters for interface and protocol, persisting events in SQLite or PostgreSQL, adding user authentication, and creating configurable thresholds. A later version could calculate rolling baselines and evaluate an anomaly-detection model on labelled data. Any automated containment should be added only after testing, with explicit authorisation, audit logs and a safe rollback path.

NETSENTRY Lite combines a local HTTP dashboard, synthetic traffic generation and a basic view of established outbound connections. It provides three repeatable scenarios and explains its process/port-based verdicts, helping students connect network concepts with introductory security monitoring. The most important distinction is between simulation and observation: the generated threat events and blocking messages are simulated, while the optional connection table uses operating-system metadata. The project is useful as a learning prototype, but packet capture, validated threat intelligence, persistent records and production-grade detection remain outside its current scope.

NETSENTRY demonstrates the basic path from network-event data to a rule-based alert and a reviewable incident. Its dashboard and simulation lab help make the detection process easier to follow, while the dry-run response keeps the demonstration safe. The project currently proves the design and the synthetic-event workflow; live packet monitoring and production-grade detection remain future work. Final evaluation should include screenshots and recorded outcomes from the running application.

References

The following protocol specifications and official documentation support the networking concepts and implementation libraries used in the updated code.

[1] W. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Internet Engineering Task Force, Aug. 2022. https://www.rfc-editor.org/rfc/rfc9293 (accessed 21 Oct. 2026).

[2] J. Postel, "User Datagram Protocol," RFC 768, Internet Engineering Task Force, Aug. 1980. https://www.rfc-editor.org/rfc/rfc768 (accessed 21 Oct. 2026).

[3] MDN Web Docs, "An overview of HTTP," https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview (accessed 21 Oct. 2026).

[4] Python Software Foundation, "http.server — HTTP servers," Python 3 documentation, https://docs.python.org/3/library/http.server.html (accessed 21 Oct. 2026).

[5] psutil developers, "psutil documentation: network connections and process APIs," https://psutil.readthedocs.io/ (accessed 21 Oct. 2026).

AI Usage Declaration

AI tools were used to assist with editing and organising report text. The team is responsible for checking the report against the supplied source code, running the application, capturing genuine screenshots and confirming test outcomes before submission. Illustrative images and pending test entries must not be represented as completed experimental results.

Declaration: We take responsibility for the accuracy of this report and understand the submitted implementation and results. Sources and assistance used have been acknowledged.

Student Name	PRN	Signature
Ranaraj Parit	1272250843	
Rajan Patil	1272250847	
Tejas Jadhav	1272250848	

Appendices

Appendix A summarises the updated implementation. The supplied Python script contains the HTTP server, local IP detection, optional psutil connection inspection, process/port heuristic, synthetic event generator, API handlers and embedded HTML/CSS/JavaScript dashboard. psutil is optional for the simulated-event tab but required for the real-connection list. The complete source should be submitted separately with the project files.

Appendix B should contain genuine screenshots from the updated application: the dashboard after startup, the event stream while live simulation is active, at least one scenario result, the Real-Time Device Behavior table when connections are available, and any relevant terminal output. Any illustrative images currently in the report must be replaced by actual captures before final submission.

Appendix B Additional Evidence

Suggested evidence to add: the dashboard loaded at http://127.0.0.1:8000, a port-scan scenario result, SSH login-failure records, a data-exfiltration event, the reset result, and the optional real-connections table. Include date/time and explain what each screenshot demonstrates. Do not describe synthetic "blocked" records as real firewall enforcement.

The contribution rows are provisional and should be edited by the group to reflect the work actually completed by each student.

Confirm the contribution entries against the source files, testing notes and presentation responsibilities before submission.

Student Name and PRN	Work Completed	Evidence or Deliverable
Ruturaj Parit — 1272250843	Implementation and Documentation	Update with actual source changes, testing or documentation.
Rajas Patil — 1272250847	Implementation and Documentation	Update with actual source changes, testing or UI work.
Tejas Jadhav — 1272250848	Implementation and Documentation	Update with actual testing, report or presentation work.